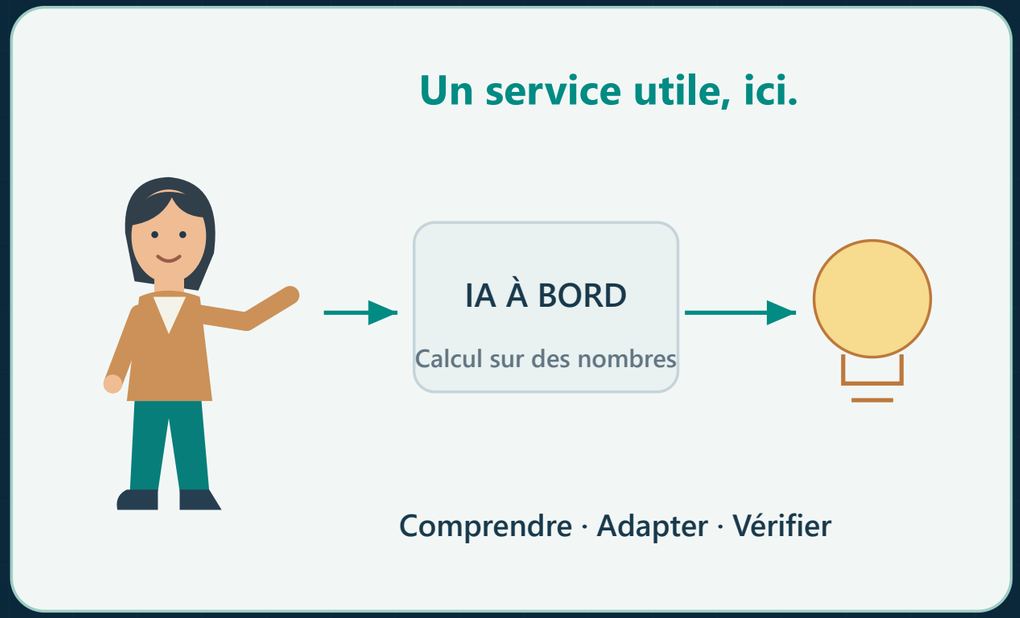
UN BESOIN · DES OBSTACLES · DES IDÉES

Alice veut emporter son IA.

IA pour les systèmes embarqués · Une histoire en six actes · 3 heures

Roland Cédric TAYO

Alice est fictive. Les travaux et leurs auteurs sont documentés.



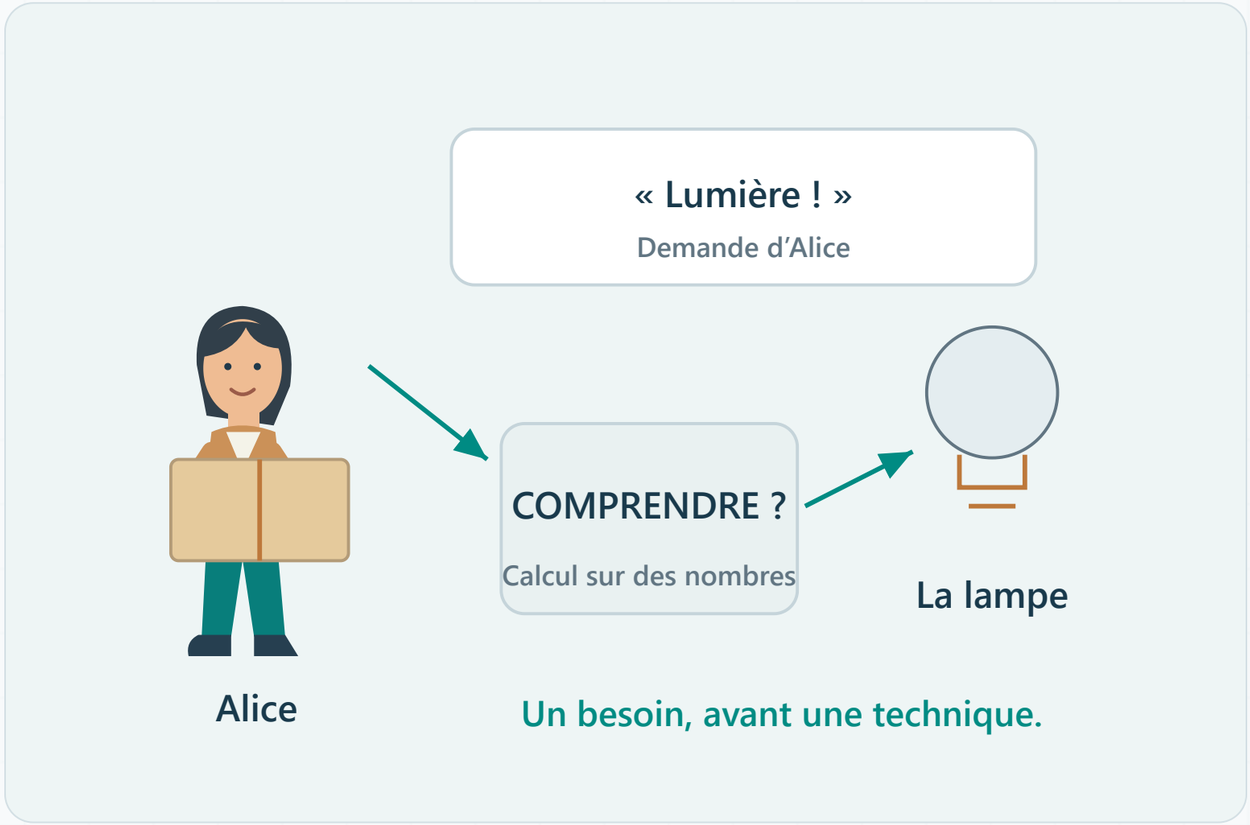
Les mains occupées. Une lampe à allumer.

Avant le modèle, une personne attend un service.

Alice

« Je veux allumer cette lampe sans poser ce que je porte. »

- **Le besoin :** agir sans utiliser ses mains.
- **Une option :** reconnaître le mot « lumière ».
- **À vérifier :** est-ce plus utile qu'un bouton ou une règle ?



Le même mot.
Jamais exactement le même son.

Alice essaie de comparer les sons à un enregistrement unique.

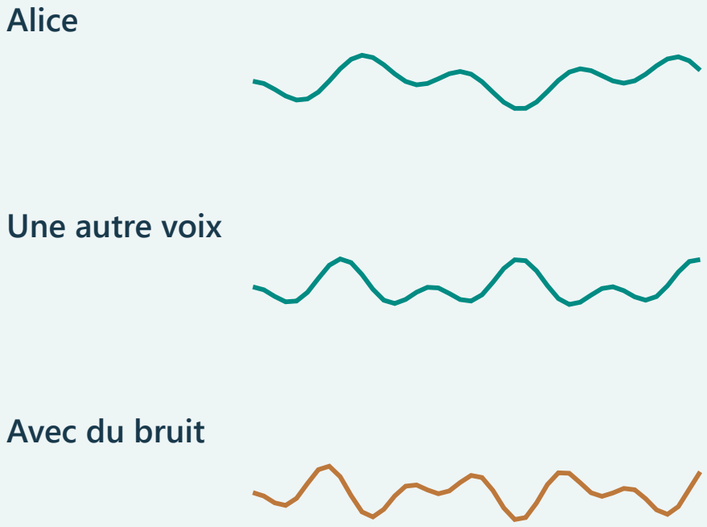
> **Alice**
> « Mon appareil comprend ma voix… mais pas celle de mon voisin. »

Les variations : voix, rythme, distance et bruit.

La difficulté : écrire toutes les formes possibles à la main.

L'idée : apprendre une relation à partir d'exemples, puis la tester.





Son → caractéristiques → modèle → mot ou refus

Bien avant Alice : faire agir une machine.

1966–1972 · SRI · Shakey perçoit une pièce et planifie des actions.

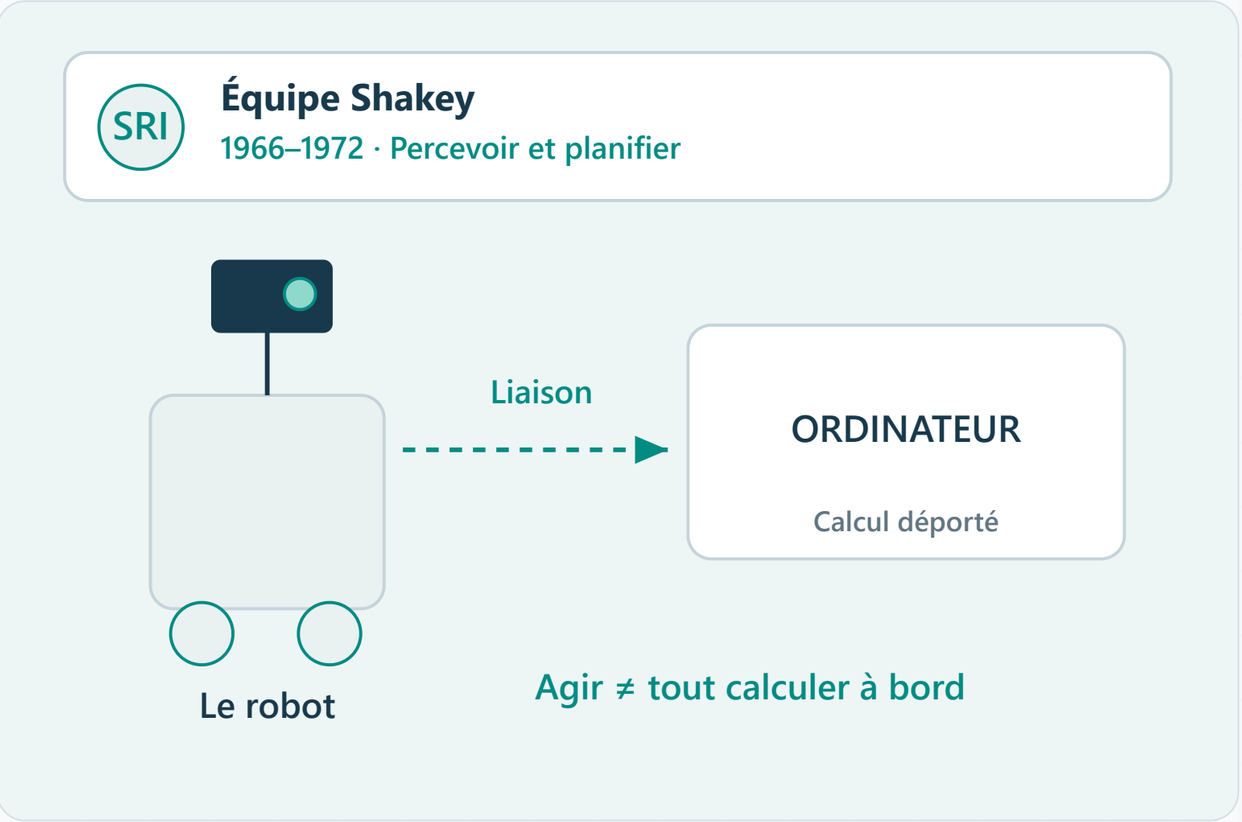
Alice

« Une machine peut agir… mais où est son calcul ? »

Le besoin : relier observation, raisonnement et déplacement.

La réponse : capteurs, représentation et planification.

La limite : agir dans le monde ne signifie pas tout calculer à bord.



1989 · Dean Pomerleau fait apprendre une direction.

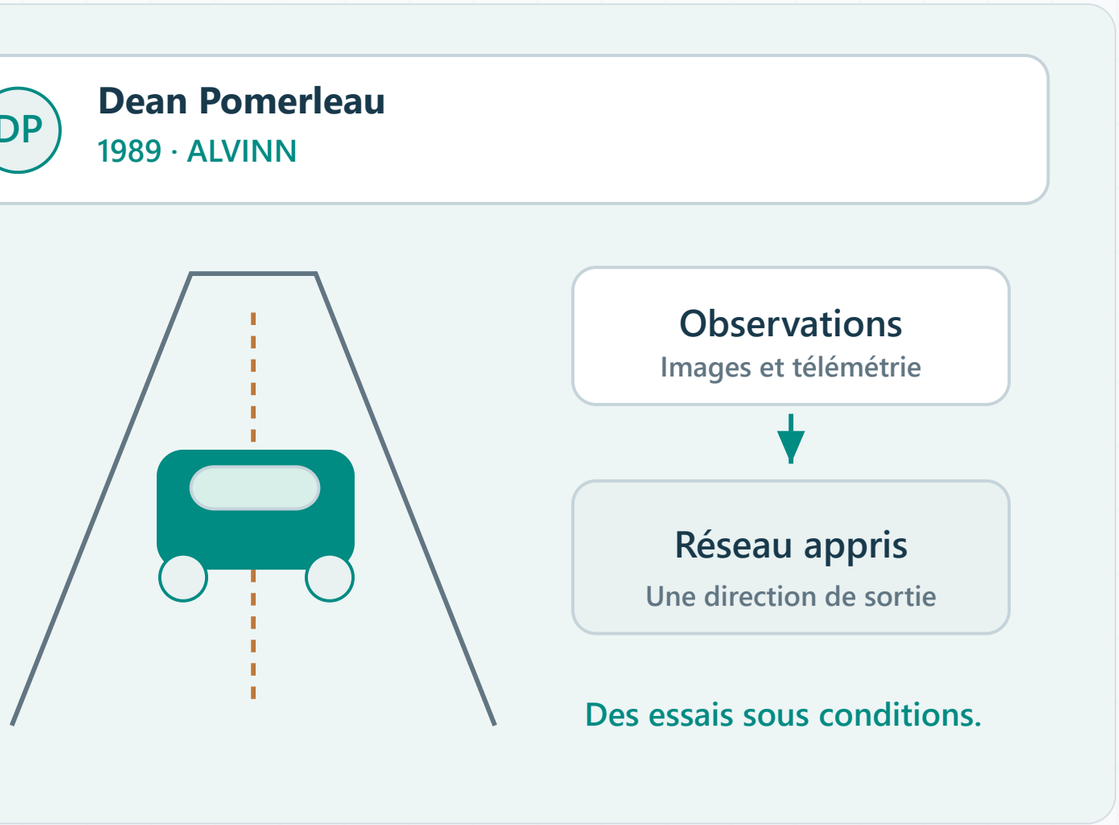
ALVINN : des observations de la route deviennent une direction à suivre.

> **Alice**
> « Notre tâche est différente, mais cette idée peut nous servir. »

Obstacle : relier des images variables à une sortie utile.

Idée : ajuster les poids d'un réseau sur des exemples.

Limite : des tests réussis sous conditions, pas une conduite autonome générale.



Questions 1 / 6 · Pourquoi envisager un apprentissage ?

La même commande présente des sons différents selon les personnes et le bruit.

?

À vous de faire avancer l'histoire.

Expliquez ce qui varie avant de choisir. Une IA reste une option à comparer.

- Une règle ou un bouton pourrait-il suffire ?
- Quelles personnes et quels bruits faut-il inclure dans le test ?

A Un réseau peut apprendre des régularités ; il faut vérifier sa qualité sur des exemples distincts.

B Un réseau rend inutile la définition du service et du coût des erreurs.

C Un seul enregistrement suffit dès que le réseau a beaucoup de paramètres.

Alice ouvre la boîte noire.

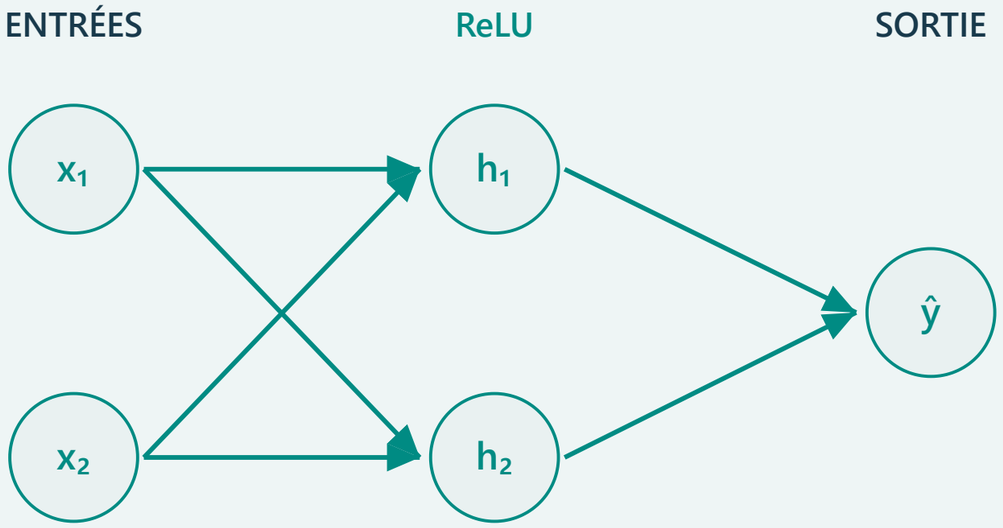
Maquette de calcul : 2 entrées → 2 neurones cachés → 1 sortie. Ce jouet ne reconnaît pas un mot.

> **Alice**
> « Avant de miniaturiser le modèle, je veux comprendre ce qu'il fait. »

Poids : coefficients qui multiplient les entrées ; biais : décalages ajoutés.

ReLU : max(0,z), pour dépasser une simple relation linéaire.

Une inférence : appliquer les calculs avec les poids fixés.



6 poids + 3 biais = 9 paramètres

2 × 2 + 2 × 1 = 6 MAC, biais exclus

Des nombres entrent. Une prédiction sort.

Modifier les entrées change les activations, sans modifier les poids.

Alice
« Montre-moi le chemin de chaque nombre. »

Entrées : $x_1=1$ et $x_2=2$; les biais sont nuls ici.

Activations : $h_1=\max(0,x_1-x_2)$; $h_2=\max(0,0{,}5x_1+0{,}5x_2)$.

Prédiction : $\hat{y}=2h_1+h_2$. Calculer avant de déplacer les curseurs.

INFÉRENCE · POIDS FIXES

x_1 1

x_2 2

h_1	h_2
0	1,5

$\hat{y} = 1{,}5$

h_1, h_2 : activations après ReLU. Les biais sont nuls ici ; aucun poids ne change.

La prédiction est fausse. Comment apprendre ?

Alice fixe une cible c=2. On corrige un seul poids de sortie w, les autres restant fixes.

A Alice

« Dans quel sens changer le poids, et de combien ? »

Perte : $L=(\hat{y}-c)^2$ mesure l'erreur sur cet exemple.

Gradient : $\partial L/\partial w=2(\hat{y}-c)h_2$ indique la pente ; ici −1,5.

Correction : $w \leftarrow w-\eta\times$gradient ; η est le pas d'apprentissage.

UNE CORRECTION · UN SEUL POIDS

Pas η **0,1**

Poids après correction	Nouvelle sortie ŷ
1,15	**1,725**

Perte avant	Perte après
0,25	**0,075625**

La perte diminue sur cet exemple.

h_2 = 1,5 et cible c = 2 sont fixes. Une correction n'est pas un entraînement complet.

Alice reconnaît ses exemples. Et les autres ?

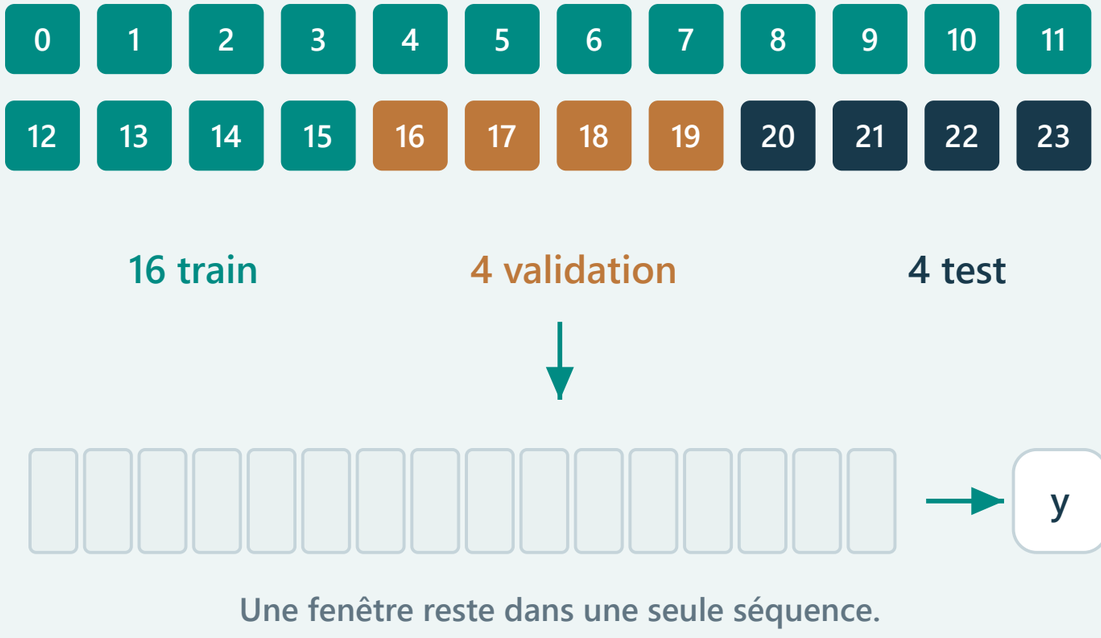
Réussir sur ce que l'on a déjà vu ne démontre pas la généralisation.

> **A** Alice
> « Mon modèle a-t-il appris la tâche, ou seulement mes exemples ? »

Apprentissage : régler poids et normalisation, c'est-à-dire le centrage et l'échelle.

Validation : choisir les réglages ; test réservé : évaluer après les choix.

Pour un flux : séparer les séquences avant de former des fenêtres voisines.



Une fenêtre reste dans une seule séquence.

Questions 2 / 6 · Suivre et compter le calcul.

Même réseau jouet, entrée $(x_1,x_2)=(2,1)$. Chaque neurone calculé possède un biais.

?

À vous de faire avancer l'histoire.

Écrivez h_1, h_2, puis $\hat{y}$. Comptez ensuite paramètres et MAC, biais exclus.

- Qu'est-ce qui change pendant l'inférence ?
- Pourquoi une perte d'apprentissage faible ne suffit-elle pas ?

A $\hat{y}=3,5$; 9 paramètres ; 6 MAC.

B $\hat{y}=1,5$; 6 paramètres ; 9 MAC.

C $\hat{y}=3,5$; 9 paramètres ; 9 MAC.

Chez Alice, le serveur semble tout résoudre.

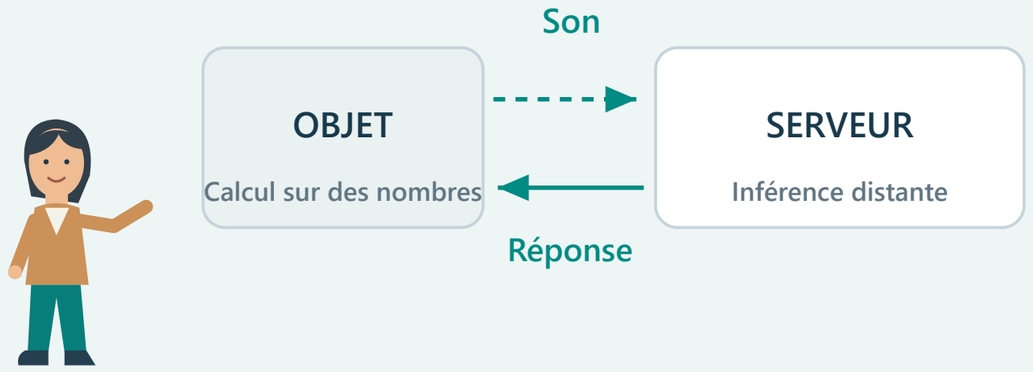
Schéma de fonctionnement : envoyer le son, calculer à distance, recevoir une décision.

> **Alice**
> « Tant que le serveur répond, ma lampe fonctionne. »

À distance : ressources de calcul et mises à jour centralisées.

À ajouter : transmission, attente et retour de la réponse.

Dépendance : chaque commande doit pouvoir joindre le service distant.



Dépendre du réseau change le service.

Schéma logique : aucun serveur contacté, aucun délai mesuré.

Même mot. Même modèle. Cette fois, rien ne se passe.

La connexion tombe. Le modèle distant n'a pas changé, le service rendu si.

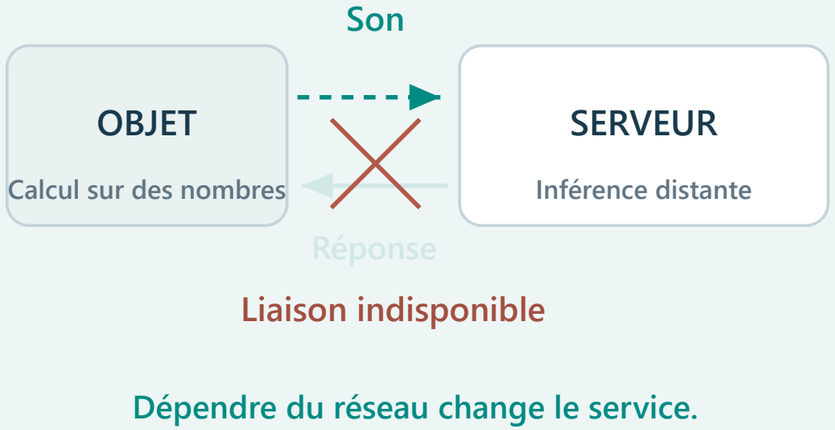
> **Alice**
> « Je n'ai pas besoin d'un meilleur score : j'ai besoin que cela fonctionne ici. »

Cause : la chaîne dépend d'une liaison devenue indisponible.

Question : quelle fonction doit rester possible pendant la coupure ?

Piste : rapprocher l'inférence des données et prévoir un repli.





Dépendre du réseau change le service.

Schéma logique : aucun serveur contacté, aucun délai mesuré.

Embarquer quoi ? Et à quel endroit ?

Le lieu de l'inférence et le lieu de l'apprentissage sont deux décisions différentes.

Alice

« L'appareil peut-il fonctionner seul sans tout apprendre tout seul ? »

Dans l'objet : réponse à bord, sous ses contraintes.

Passerelle : calcul proche, mais liaison locale nécessaire.

Hybride : entraînement ou mises à jour distants, fonction locale définie.

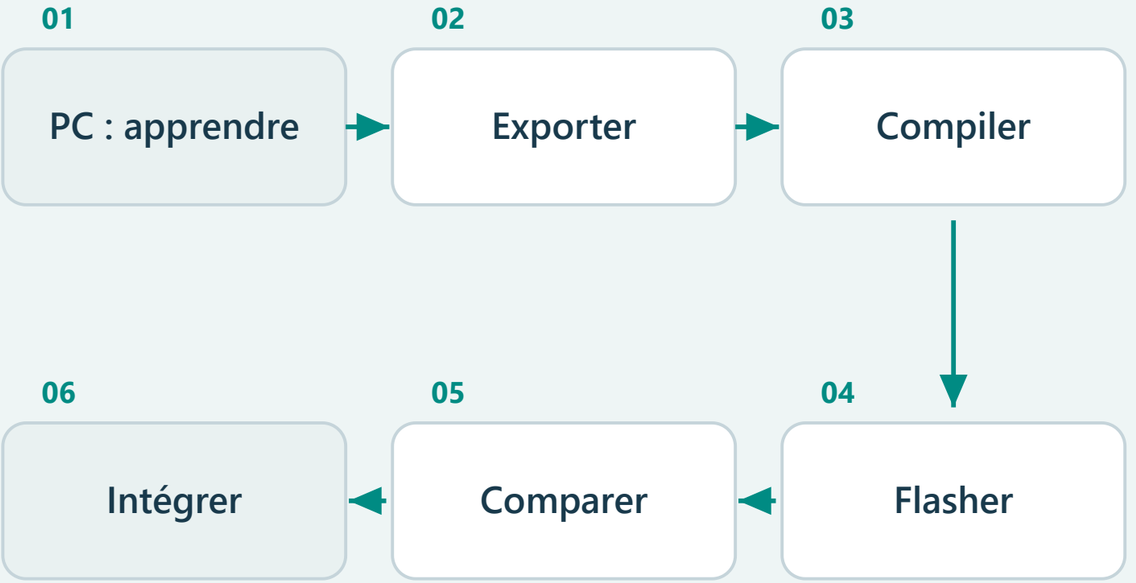


Du PC à l'appareil : faire voyager le bon contenu.

Alice emporte les calculs, les poids et le prétraitement, pas son ordinateur d'apprentissage.

> **Alice**
> « Mon programme démarre. Mais calcule-t-il encore la même chose ? »

- **Figer :** poids, formes, ordre des variables et normalisation.
- **Construire :** compiler ou installer pour la vraie cible.
- **Comparer :** mêmes entrées, sorties et tolérance numérique expliquée.



Questions 3 / 6 · Alice veut une commande autonome.

Deux exigences : fonctionner sans Internet et ne pas faire sortir l'audio de l'objet.

?

À vous de faire avancer l'histoire.

Dessinez le trajet des données et vérifiez les deux exigences.

- Quelles fonctions peuvent encore utiliser un serveur ?
- Quel repli faut-il prévoir si la reconnaissance se trompe ?

A Reconnaître dans l'objet ; télécharger des mises à jour lorsque la liaison revient.

B Reconnaître sur une passerelle Wi-Fi locale, sans utiliser le cloud.

C Envoyer seulement de courts extraits audio au serveur.

Dix minutes.
La prochaine difficulté attend.

Alice veut maintenant faire entrer le modèle dans sa petite boîte.

> **Alice**
> « Il ne reste plus qu'à le mettre dedans… vraiment ? »

Déjà compris : besoin, apprentissage et dépendances.

À découvrir : mémoire, temps, énergie et preuves.



10 MINUTES

La boîte attend…

Comprendre la suite, puis la mesurer.

Alice compare des appareils, pas seulement des scores.

Un MCU, un ordinateur Linux et un accélérateur n'ont ni le même rôle ni le même budget.

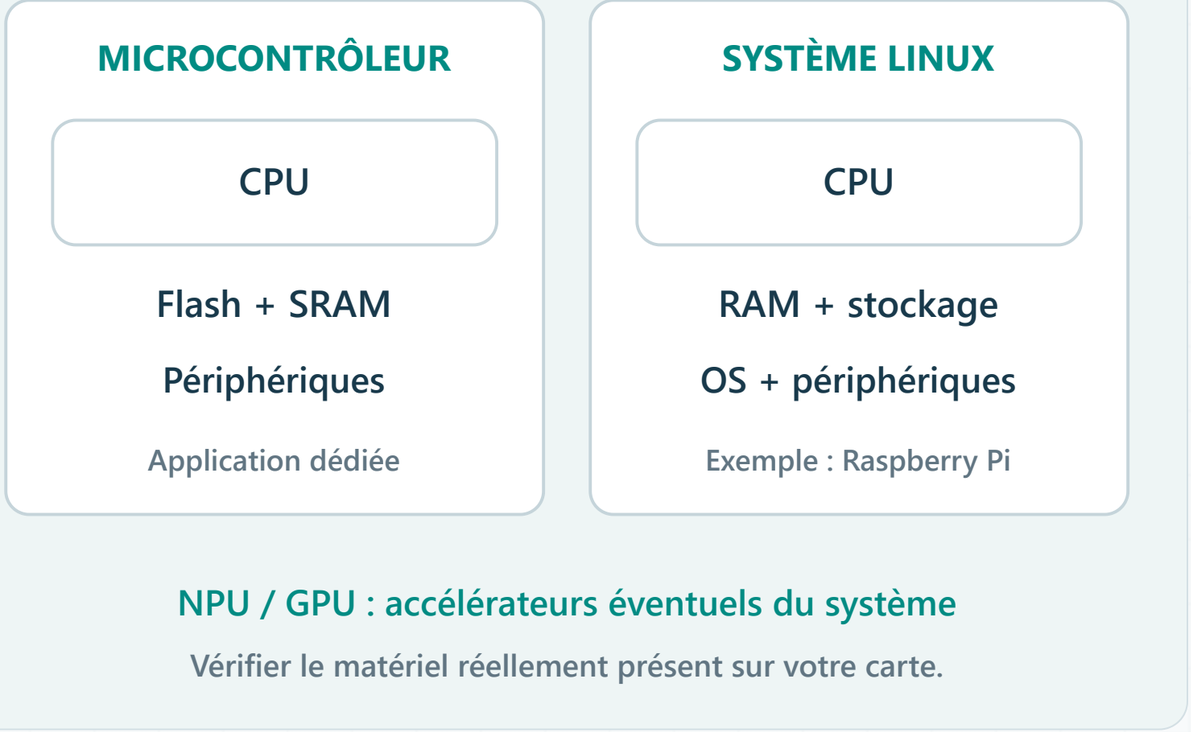
Alice

« Quelle boîte peut réellement rendre mon service ? »

MCU : CPU, mémoire et périphériques dans un petit système.

Pi / SoC : autre capacité mémoire et système logiciel.

NPU / GPU : accélérateurs éventuels ; vérifier leurs formats et opérateurs.



Le fichier entre.
Le programme déborde.

Alice avait compté les poids, mais oublié la mémoire pendant le calcul.

> **Alice**
> « Pourquoi le modèle tient-il sur disque, mais pas en fonctionnement ? »

Flash / stockage : programme et constantes selon leur placement.

RAM : entrées, activations, buffers temporaires et pile.

Pic : compter ce qui vit au même moment dans toute l'application.

EXEMPLE MCU · POIDS CONSERVÉS EN FLASH

Poids bruts	Travail SRAM
400 Ko	**120 Ko**
100 000 paramètres	activations + buffers + pile



Poids FP32 | Poids INT8 seuls

Budgets fictifs distincts. Échelles et métadonnées exclues du payload ; SRAM inchangée dans cet exemple.

2017 · Howard et ses collègues revoient les opérations.

MobileNets : alléger la vision en séparant filtrage spatial et mélange des canaux.

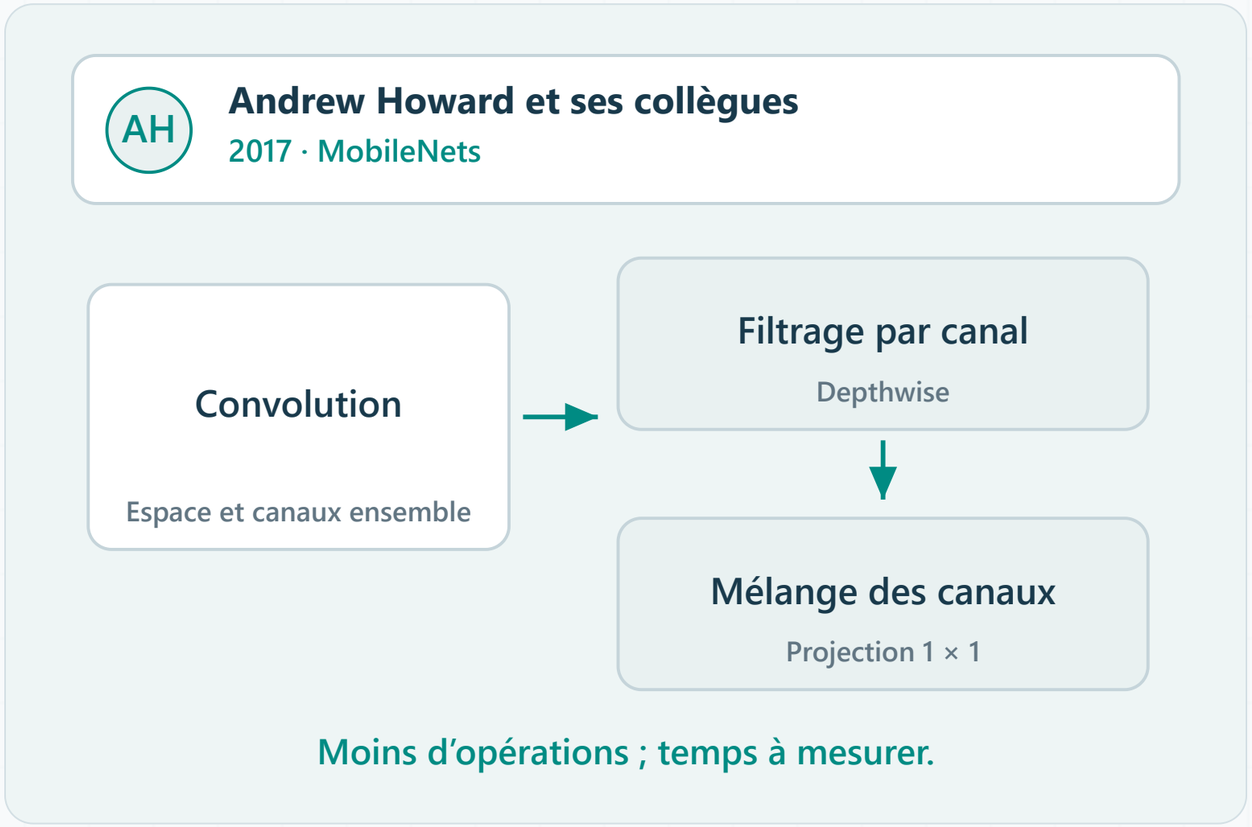
Alice

« Peut-on repenser le calcul plutôt que seulement changer la puce ? »

Obstacle : des convolutions coûteuses pour un appareil mobile.

Idée : filtrer par canal, puis mélanger avec une projection 1×1.

Limite : moins de MAC ne garantit pas le même gain de temps sur chaque cible.



2018 · Jacob et ses collègues changent les nombres.

Quantification et apprentissage adaptés à une inférence en calcul entier.

Alice

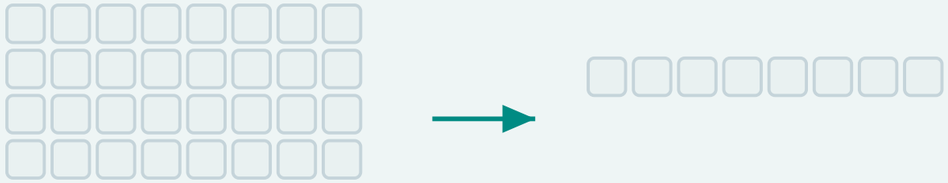
« Ai-je vraiment besoin de tous ces chiffres pour chaque poids ? »

Obstacle : des représentations et calculs flottants coûteux selon le matériel.

Idée : coder sur moins de bits, avec échelles et accumulateurs adaptés.

Condition : vérifier la qualité et les noyaux réellement exécutés.

Benoit Jacob et ses collègues
2018 · Quantification et calcul entier



Poids FP32 → Poids INT8

Qualité à vérifier

+ échelles
+ accumulateurs adaptés

Alice arrondit. Qu’est-ce qu’elle perd ?

Une grille plus grossière économise des codes, mais modifie les valeurs.

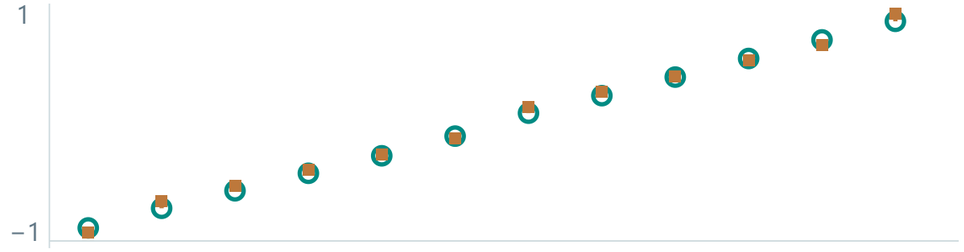
> **A** Alice
> « Quelques chiffres en moins, est-ce encore la même réponse utile ? »

Échelle s : distance entre deux valeurs représentables.

Code q : arrondi de x/s, limité à une plage entière.

Reconstruction : $\hat{x}=s\times q$; comparer l’erreur et la qualité du réseau entier.

ARRONDI SYMÉTRIQUE · EXEMPLE SUR [−1, 1]



12 valeurs illustratives

Précision des codes **4 bits**

Pas s : **0,1429**

Erreur max illustrée : **0,07**

Cercles : valeurs initiales ; carrés : valeurs reconstruites. Pas un test de qualité de réseau.

Questions 4 / 6 · Quelle mémoire faut-il libérer ?

128 Kio de SRAM. Poids : 80 Kio en Flash. En RAM, simultanés : 60 + 40 + 20 + 16 Kio.

?

À vous de faire avancer l'histoire.

Dessinez le pic et additionnez les objets RAM, puis choisissez.

- Réduire seulement les poids Flash résout-il le pic SRAM ?
- Quand deux buffers peuvent-ils partager le même espace ?

A 120 Kio : les entrées-sorties peuvent être ignorées.

B 136 Kio : le pic dépasse la capacité de 8 Kio.

C 216 Kio : tous les poids doivent toujours être copiés en SRAM.

Alice réduit les calculs. L'attente reste.

Le processeur peut attendre les données plutôt que manquer de multiplications.

Alice

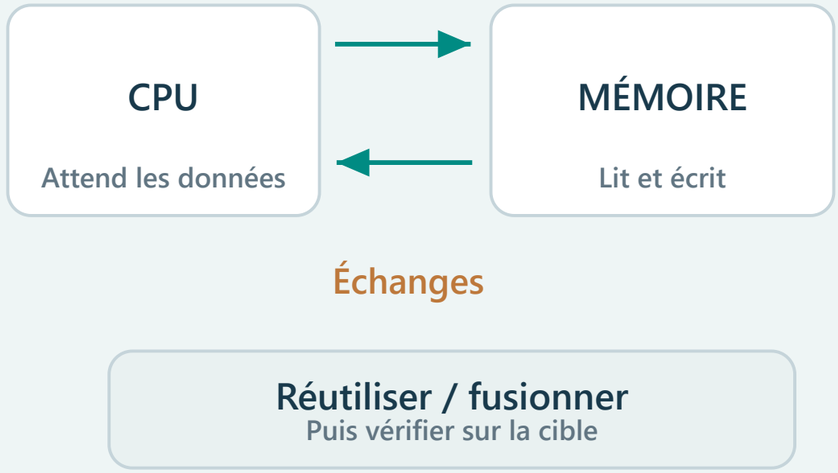
« Pourquoi le processeur attend-il alors que le calcul paraît simple ? »

Lire et écrire : des transferts qui ont un coût.

Réutiliser : garder une donnée proche et éviter des passages inutiles.

Tester : réorganisation, sorties, buffers et compilateur sur la cible.





2020 · Lin, Chen et leur équipe conçoivent les deux ensemble.

MCUNet associe une architecture compacte et son moteur d'exécution.

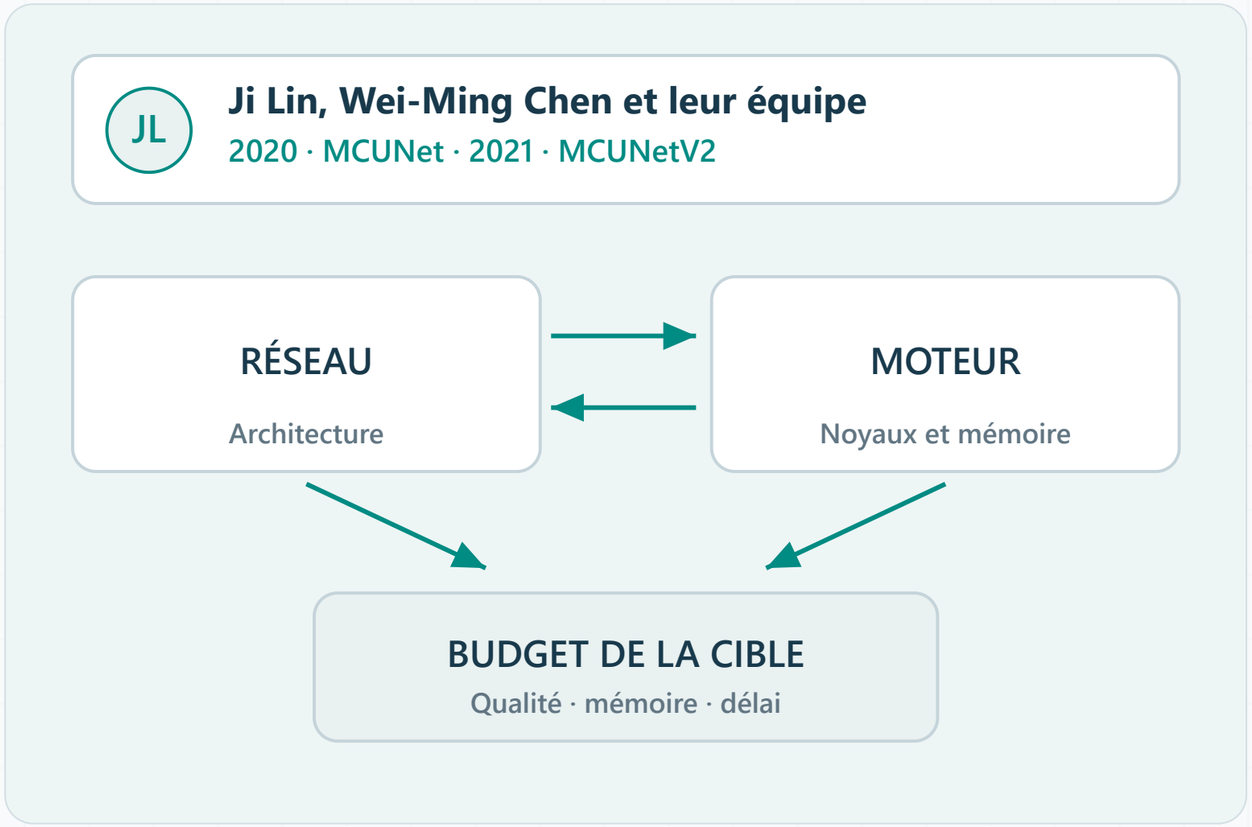
A Alice

« Le meilleur réseau dépend aussi de la manière de le faire tourner. »

Obstacle : la mémoire de travail limite l'inférence sur MCU.

Réponse : adapter réseau et ordonnancement à la cible.

Suite : MCUNetV2 découpe le travail ; examiner aussi les recalculs ajoutés.



La réponse est correcte. Mais la lampe réagit trop tard.

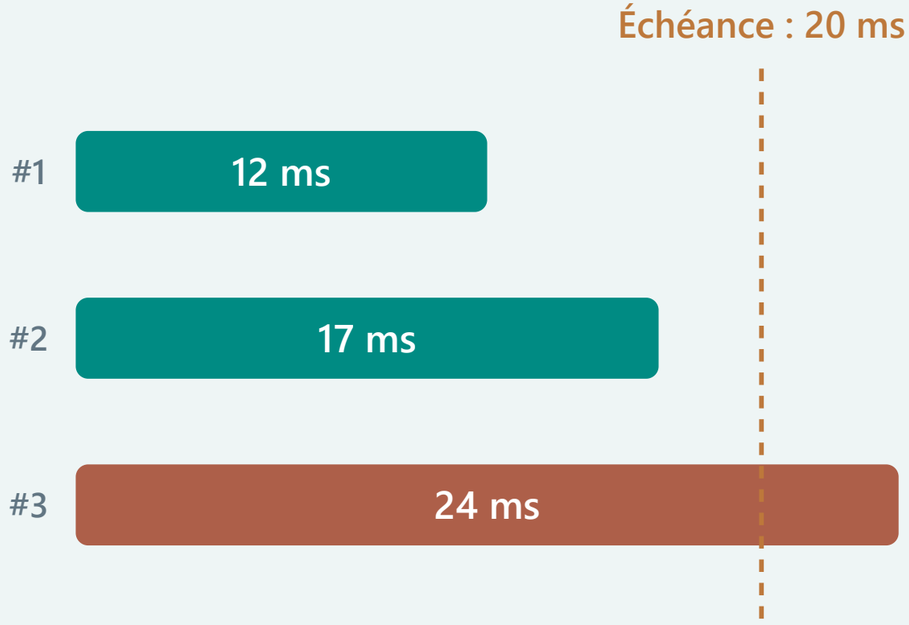
Le service attend une réponse dans un délai défini, pas seulement une bonne moyenne.

> **Alice**
> « Une réponse utile est aussi une réponse qui arrive à temps. »

Latence : délai pour une entrée ; débit : sorties par seconde.

Gigue : variabilité du délai entre les appels.

Échéance : inclure acquisition, préparation, calcul et action ; prévoir un repli.



Durées pédagogiques · le troisième appel est trop tardif.

Alice débranche le chargeur. Une autre limite apparaît.

Le calcul compte, mais aussi l'écoute, le capteur, la radio et la veille.

> **Alice**
> « Mon appareil doit-il être rapide pendant une seconde, ou utile toute la journée ? »

Puissance P : en watts ; énergie E : en joules.

À puissance constante : $E = P \times \Delta t$; 0,1 W pendant 0,02 s donne 2 mJ.

Mesure : préciser le périmètre et utiliser un instrument adapté.





Écouter | Calculer | Échanger

Batterie : toute la chaîne compte.

Une brique gagne. Le système gagne combien ?

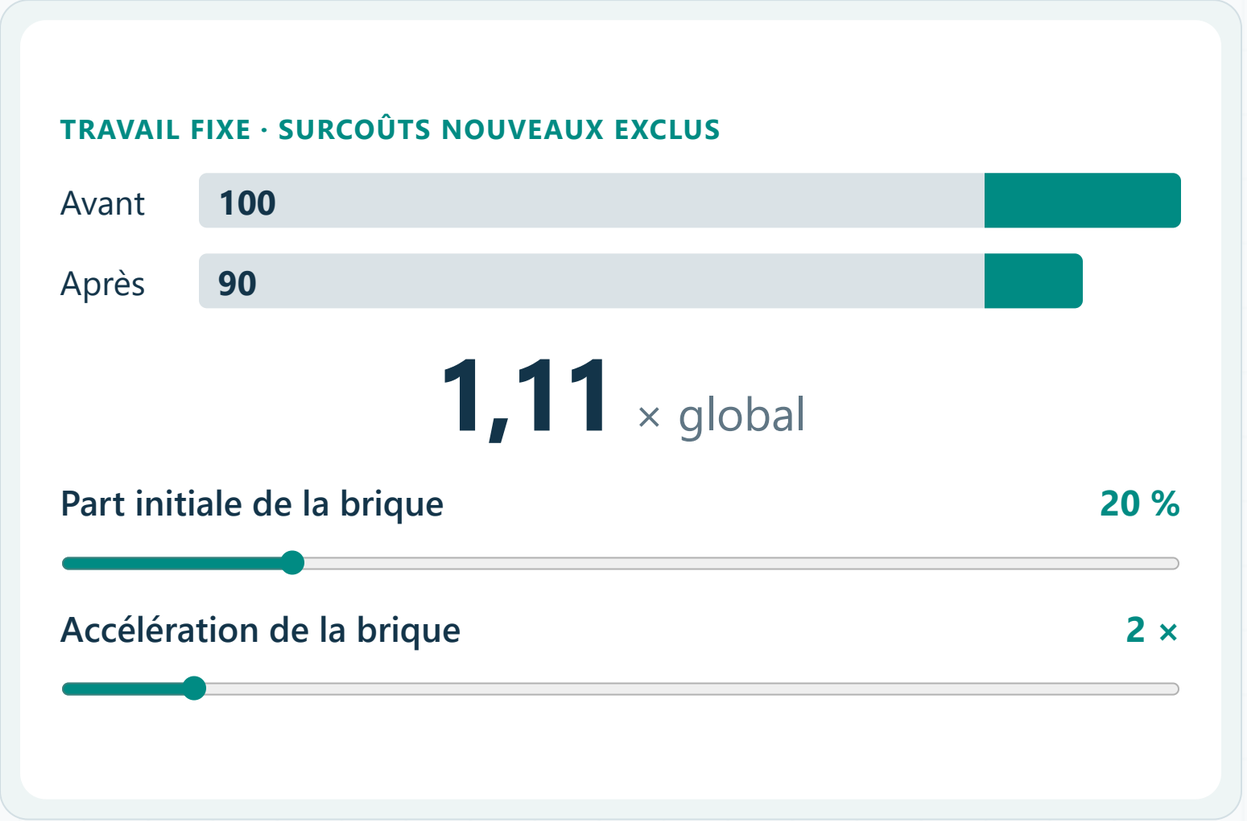
La partie inchangée limite l'accélération : loi d'Amdahl.

> **Alice**
> « Deux fois plus vite… pour quelle partie exactement ? »

p : fraction initiale du temps dans la brique ; s : son accélération.

Gain global : 1 / [(1−p)+p/s], si le reste ne change pas.

Exemple : p=20 % et s=2 donnent 1,11×, soit 10 % du temps économisé.



Questions 5 / 6 · Alice annonce un gain de 2×.

Avant : 100 ms au total, dont 20 ms dans la brique. Cette brique devient deux fois plus rapide.

?

À vous de faire avancer l'histoire.

Reconstituez les deux parties du temps. Le reste est inchangé.

- Que devient le résultat si une conversion ajoute 12 ms ?
- Ces durées permettent-elles de calculer l'énergie consommée ?

A 50 ms : toute l'application devient deux fois plus rapide.

B 90 ms : le système économise 10 % de sa durée.

C 80 ms : la brique optimisée ne coûte plus rien.

« Plus rapide », dit Alice.
« Montre-le », répond la classe.

Micro-expérience indépendante : deux écritures de la même somme, dans ce navigateur.

> **Alice**
> « Je veux un résultat que quelqu'un d'autre puisse vérifier. »

Avant le chrono : vérifier que les sorties sont égales.

Protocole : échauffement, sept paires alternées, médiane et min–max.

Trace : conserver le CSV ; aucune durée de carte ni énergie n'est déduite.

NAVIGATEUR ACTUEL · SOMME DE 4 096 VALEURS

Pas encore mesuré. Aucune durée préremplie.

Par somme	Médiane	Min–max
Boucle	—	—
reduce	—	—

Résultat vérifié avant et après. Données : Float64Array, 32 Kio ; ceci n'est pas un pic RAM système.

Les mesures varient avec le navigateur et sa charge. Aucune énergie ni durée sur carte n'est déduite.

La démonstration réussit. La vraie vie change les entrées.

Une voix inconnue, du bruit ou un redémarrage peuvent changer le comportement.

Alice

« Et quand il entend autre chose que “lumière” ? »

Robustesse : tester bruit, saturation, valeurs absentes et décalage des données.

Maîtrise : refuser, signaler ou revenir à une commande manuelle.

Sécurité : contrôler les accès et les mises à jour ; local ne signifie pas invulnérable.



Voix connue
Cas nominal

Bruit seul
Entrée non prévue

Redémarrage
État à reconstruire

Tester le refus et le repli.

Les étudiants prennent la suite de l'enquête.

Le TD utilise un petit Mixer de prédiction, pas un système vocal fourni.

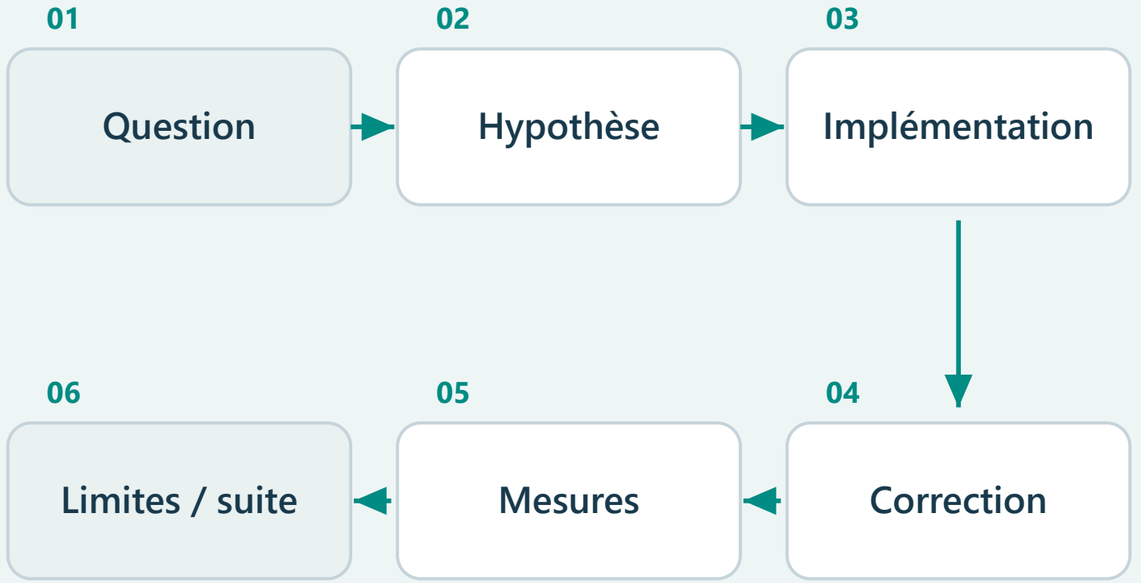
Alice

« Dans quelles conditions notre idée aide-t-elle vraiment ? »

Construire : trois signaux, seize instants ; écrire calculs et apprentissage.

Enquêter : une projection dense, poids fixés, correction puis mesures PC/carte.

Tracer : commit → configuration → binaire → CSV → figure → conclusion.



La conclusion peut être négative.

Aujourd'hui : plusieurs pistes, pas une seule course aux paramètres.

Le besoin d'Alice reste petit ; les modèles et les matériels continuent d'évoluer.

Alice

« Ai-je besoin d'un grand modèle de langage pour reconnaître un mot ? »

Architecture : MobileNetV4 étudie plusieurs cibles mobiles.

Représentation : AWQ examine la quantification des poids de LLM.

Adaptation : apprentissage local spécialisé ; choisir selon mémoire, données et usage.



Architecture
MobileNetV4 · 2024

Représentation
AWQ · 2024

Adaptation locale
Mises à jour ciblées

Le besoin choisit l'échelle du modèle.

Questions 6 / 6 · Quelle boîte Alice peut-elle retenir ?

Exercice fictif : qualité ≥96 %, RAM ≤100 Kio, p95 observé ≤20 ms et énergie ≤3 mJ par décision.

Valeurs fictives · périmètre commun

Modèle	Qualité	Kio	ms[1]	mJ
A	97 %	120	12	2,4
B	96 %	96	18	2,8
C	95 %	72	14	1,8

[1] p95 sur 500 essais fictifs. Énergie : acquisition + calcul + action, hors veille.

Choisissez avec les quatre contraintes. Proposez ensuite un test qui pourrait invalider ce choix.

- Un p95 observé garantit-il tous les délais futurs ?

A A : sa qualité suffit à accepter son dépassement RAM.

B B : les seuils donnés sont respectés ; la robustesse et les limites restent à vérifier.

C C : sa faible énergie compense automatiquement sa qualité sous le seuil.

L’histoire ne se termine pas par « ça marche ».

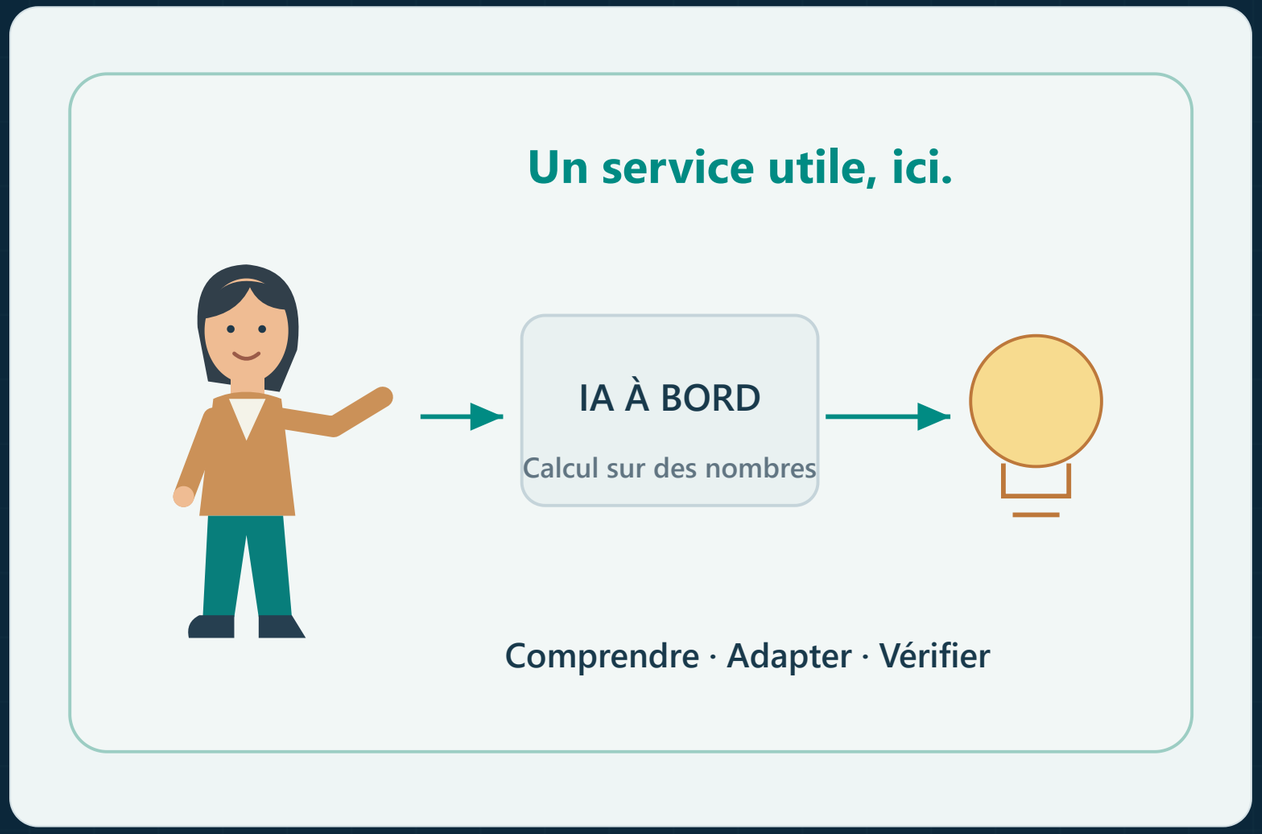
Elle se termine par : « Voilà ce qui marche, dans ces conditions, et ce qui reste à comprendre. »

> **Alice**
> « Je peux expliquer mon choix, le mesurer et reconnaître ses limites. »

Le besoin : un service utile, pas un modèle choisi pour son nom.

Le compromis : qualité, mémoire, délai et énergie dans un système réel.

La suite : une question précise, des tests et des résultats reproductibles.



La convolution réutilise un petit motif.

Une convolution séparée découpe le filtrage spatial et le mélange des canaux.

Classique : chaque sortie combine espace et canaux.

Depthwise + 1×1 : filtrer chaque canal, puis les mélanger.

À vérifier : moins de MAC peut encore produire plus de transferts.



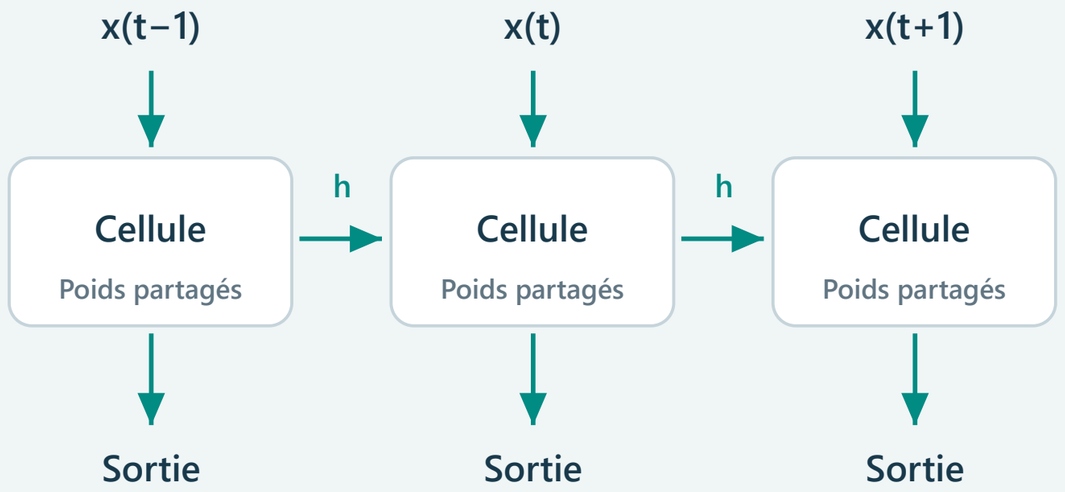
Un état mémorise une partie du passé.

Un réseau récurrent met à jour un petit état à chaque échantillon.

État h : résumé réutilisé au pas suivant.

Intérêt : traiter un flux sans garder tout son historique.

Limite : dépendance séquentielle, dérive et temps de mise en régime.



État transmis ≠ historique complet stocké

L’attention choisit quelles relations regarder.

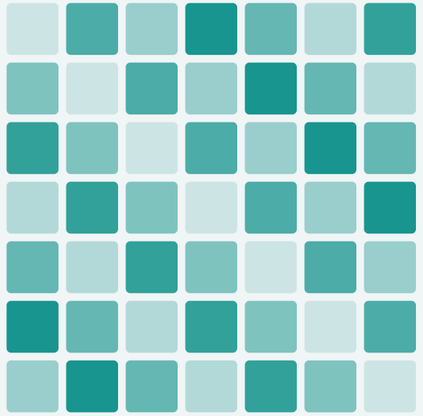
Au lieu d’un mélange fixe, les relations dépendent du contenu de l’entrée.

Q, K, V : requêtes, clés et valeurs calculées par projections.

Comparer : associer chaque position à d’autres positions.

Coût : une matrice naïve de scores grandit comme le carré de la longueur.

Scores entre positions



→

Normaliser puis mélanger

$\text{softmax}(QK^T / \sqrt{d})\, V$

^{T} : transpose · d : dimension d’une tête

Doubler la longueur → 4× plus de scores naïfs

Carte d’attention illustrative ; aucune sortie de modèle mesurée

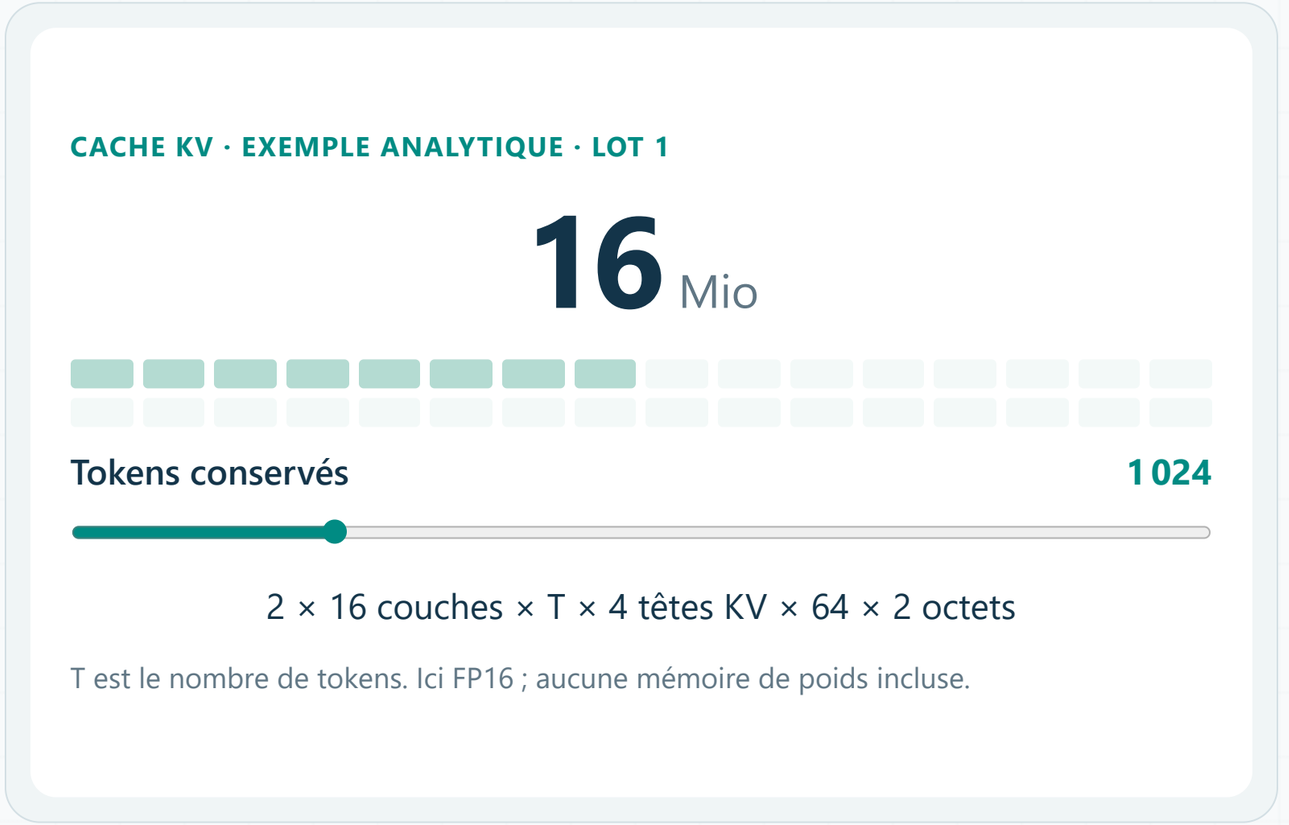
Le contexte consomme aussi de la mémoire.

Un cache KV évite de recalculer d'anciennes clés et valeurs lors de la génération.

Préremplissage : traiter la consigne initiale.

Décodage : produire ensuite un token à la fois.

Conséquence : réduire les poids ne réduit pas automatiquement le cache.



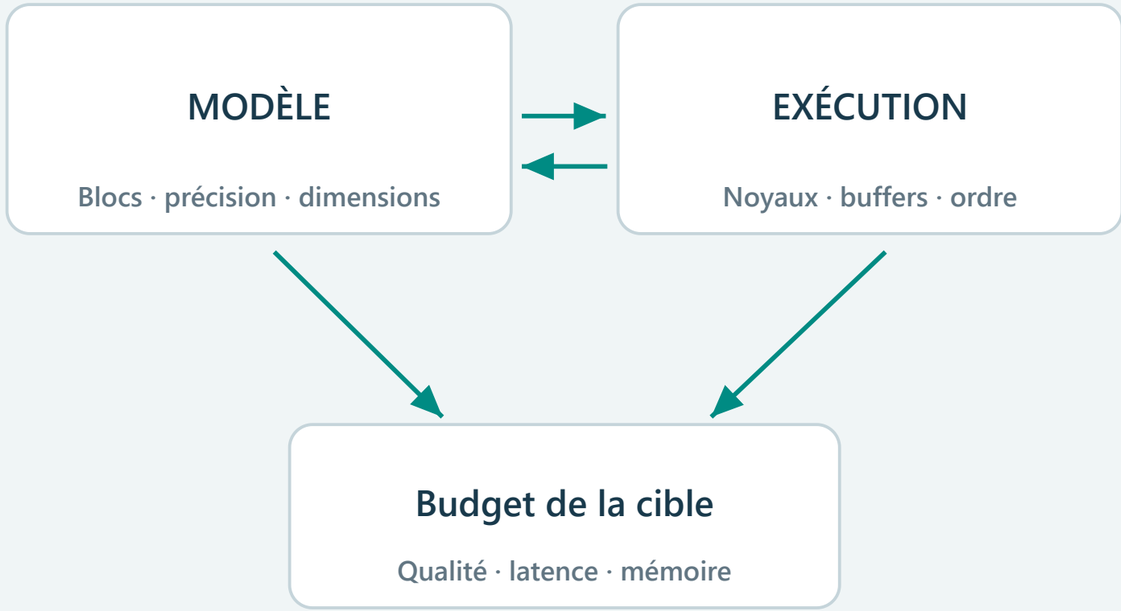
Deux recherches : déplacer la contrainte au bon endroit.

Architecture et exécution se répondent ; une seule métrique masque le compromis.

MCUNetV2 · 2021 : calculer par morceaux pour réduire le pic mémoire.

MobileNetV4 · 2024 : concevoir des blocs efficaces pour plusieurs cibles mobiles.

Question ouverte : le gain reste-t-il vrai sur notre cible et notre tâche ?



Calibrer, entraîner, puis comparer le vrai calcul.

La meilleure précision n'est pas forcément uniforme dans tout le réseau.

PTQ : quantifier après entraînement, avec calibration si nécessaire.

QAT : simuler les effets de quantification pendant l'apprentissage.

AWQ · 2024 : protéger les canaux sensibles à partir des activations.



Préserver ce qui change le plus la sortie

Puis vérifier qualité et noyau réellement exécuté.

Supprimer, simplifier, ou apprendre à imiter.

Compresser un fichier ne suffit pas à accélérer son exécution.

Élagage : retirer poids ou canaux ; exploiter réellement la structure obtenue.

Distillation : entraîner un petit élève à reproduire un enseignant.

Choix d'architecture : chercher un compromis compatible avec la cible.



Le noyau doit exploiter la structure restante.

Distillation : enseignant → petit élève à entraîner

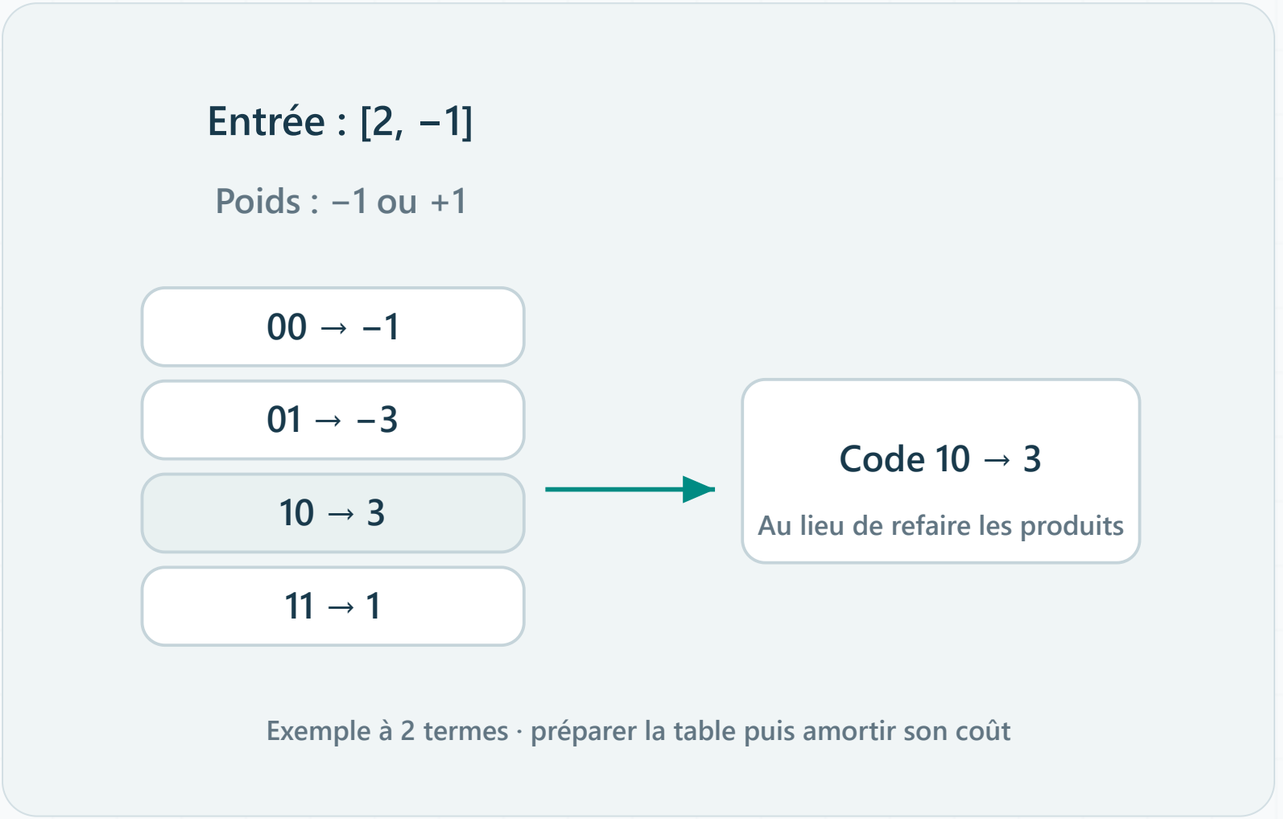
T-MAC · Remplacer un calcul par une consultation de table.

Jianyu Wei et al. · EuroSys 2025 · Une idée système pour produits bas bits sur CPU.

Problème : les poids bas bits peuvent coûter cher à déquantifier.

Idée : préparer des combinaisons, puis les retrouver par indices.

Limite : la table et ses accès doivent être moins coûteux que le calcul évité.



« Petit modèle de langage » ne signifie pas « modèle pour MCU ».

Le mot petit dépend du point de comparaison.

> **1,7 milliard de paramètres à 4 bits :** environ 850 Mo de poids bruts.

> **À ajouter :** métadonnées, activations, cache KV et moteur.

> **À décider :** besoin réel de génération, ou classification spécialisée ?

1,7 milliard × 4 bits ÷ 8

Payload : 850 000 000 octets ≈ 850 Mo

↓

+ cache + activations + moteur

Une taille de poids n’est pas un budget système.

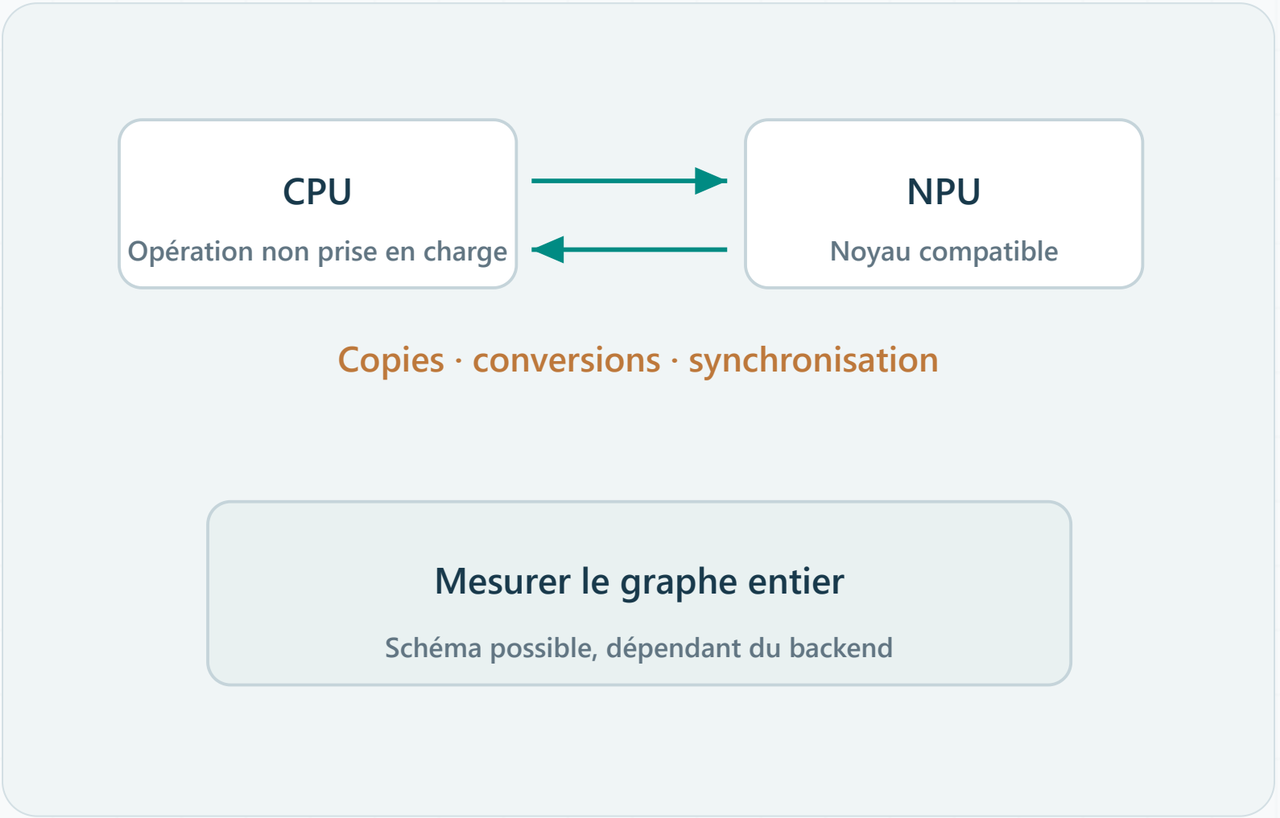
Veille 2025 · Le NPU ne gagne que si le graphe lui convient.

T-MAN · Wei, Li et al. · Prépublication : lire ce qui est accéléré, et ce qui ne l'est pas.

> **Problème :** des opérations périphériques limitent un accélérateur spécialisé.

> **Piste :** adapter tables, tuilage et pipeline.

> **À contrôler :** préremplissage, décodage, transferts et comparaison de départ.



Veille 2026 · Deux voies, deux niveaux d'intervention.

Des pistes actuelles à examiner ; aucune promesse universelle de performance.

FastGRNN · Kizilates : très petite récurrence, tables et validation sur MCU.

RISC-V · Kumar et al. : instructions et compilation pour Transformers compacts.

Regard critique : mise en régime, graines, mémoire, aire et protocole.

JUIN 2026

FastGRNN sur MCU

Mise en régime à examiner

Emre Can Kizilates

OCTOBRE 2026

Transformer sur RISC-V

Instructions + compilation
Aire et énergie à compter

Ajay Kumar M et al.

Deux prépublications : hypothèses et périmètres à vérifier.

Sources et limites de lecture · 1/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R01 ALVINN: An Autonomous Land Vehicle In a Neural Network ↗
Dean Pomerleau · 1989 · Repère historique · publication CMU / actes NeurIPS
Relier perception apprise et action à bord d'un véhicule.

R02 MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications ↗
Andrew G. Howard et al. · 2017 · Article fondateur · arXiv 1704.04861
Séparer convolution spatiale et mélange des canaux pour réduire le calcul.

R03 Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ↗
Benoit Jacob et al. · 2018 · CVPR 2018 · prépublication 2017
Co-concevoir quantification, apprentissage et noyaux entiers.

R04 MCUNet: Tiny Deep Learning on IoT Devices ↗
Ji Lin, Wei-Ming Chen et al. · 2020 · NeurIPS 2020
Chercher ensemble l'architecture du réseau et son moteur d'exécution.

Sources et limites de lecture · 2/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R05 MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning ↗
Ji Lin, Wei-Ming Chen, Han Cai et al. · 2021 · NeurIPS 2021
Traiter des morceaux de carte de caractéristiques pour réduire le pic mémoire.

R06 On-Device Training Under 256KB Memory ↗
Ji Lin, Ligeng Zhu et al. · 2022 · NeurIPS 2022
Rendre possible une adaptation locale par mises à jour sélectives et co-conception.

R07 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness ↗
Tri Dao, Daniel Y. Fu et al. · 2022 · NeurIPS 2022
Réorganiser le calcul d'attention pour réduire les échanges mémoire sur GPU.

R08 TSMixer: An All-MLP Architecture for Time Series Forecasting ↗
Si-An Chen et al. · 2023 · Transactions on Machine Learning Research, 2023
Mélanger les instants puis les variables avec des perceptrons multicouches.

Sources et limites de lecture · 3/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R09 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration ↗
Ji Lin, Jiaming Tang et al. · 2024 · MLSys 2024 · prépublication 2023
Utiliser des statistiques d'activation pour mieux préserver les canaux sensibles.

R10 MobileNetV4 — Universal Models for the Mobile Ecosystem ↗
Danfeng Qin et al. · 2024 · ECCV 2024
Concevoir des blocs et une recherche d'architecture adaptés à plusieurs matériels mobiles.

R11 T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge ↗
Jianyu Wei, Shijie Cao et al. · 2025 · EuroSys 2025 · prépublication 2024
Remplacer certaines opérations de produits mixtes par des tables de recherche adaptées.

R12 SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model ↗
Loubna Ben Allal et al. · 2025 · Prépublication arXiv 2502.02737
La qualité et le mélange des données comptent aussi pour les petits modèles de langage.

Sources et limites de lecture · 4/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R13 T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup ↗
Jianyu Wei, Qingtao Li et al. · 2025 · Prépublication arXiv · 14 novembre 2025
Adapter tables, tuilage et pipeline aux opérations réellement prises en charge par un NPU.

R14 From Compression to Deployment: Real-Time and Energy-Efficient FastGRNN on Ultra-Constrained Microcontrollers ↗
Emre Can Kizilates · 2026 · Prépublication arXiv · 15 juin 2026
Étudier ensemble compression, activation par table, état récurrent et déploiement physique.

R15 A Framework for Accelerating Transformer Inference on RISC-V for Edge AI ↗
Ajay Kumar M et al. · 2026 · Prépublication arXiv · 6 octobre 2026
Adapter instructions et compilation à des Transformers compacts, principalement encodeurs.

R16 Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations ↗
Apostol Vassilev, Alina Oprea et al. · 2025 · Rapport technique NIST AI 100-2e2025
Distinguer attaques sur données, entrées, modèles et vie privée.

Sources et limites de lecture · 5/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R17 Tiny Machine Learning: Progress and Futures ↗
Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Song Han · 2023 / 2024 · IEEE Circuits and Systems Magazine 2023 · dépôt arXiv 2024
Vue d'ensemble de la co-conception et du passage de l'inférence à l'adaptation locale.

R18 MLPerf Tiny Benchmark ↗
Colby Banbury, Vijay Janapa Reddi et al. · 2021 · NeurIPS 2021 · Datasets and Benchmarks
Comparer qualité, latence et énergie avec un protocole reproductible.

R19 CMSIS-NN — Efficient neural network kernels for Arm Cortex-M ↗
Arm · Documentation consultée en 2026 · Dépôt officiel de noyaux optimisés
Les performances dépendent de noyaux adaptés à l'ISA et aux formats quantifiés.

R20 LiteRT for Microcontrollers ↗
Google AI Edge · Documentation consultée en 2026 · Documentation officielle du moteur MCU
Déployer un sous-ensemble d'opérateurs avec une allocation adaptée au microcontrôleur.

Sources et limites de lecture · 6/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R21 MLPerf Inference: Tiny ↗
MLCommons · Documentation consultée en 2026 · Méthodologie et résultats officiels
Lire conjointement qualité, scénario de mesure et configuration matérielle.

R22 Shakey the Robot ↗
SRI International · 1966–1972 · Histoire du projet · organisme concepteur
Faire percevoir un environnement et planifier des actions à un robot mobile.

R23 Edge AI ↗
NIST · Documentation consultée en 2026 · Programme de recherche officiel
Rapprocher l'inférence ou l'apprentissage des données et des usages.

Lexique · Matériel et mémoire

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

MCU

Microcontrôleur : processeur, mémoire et périphériques réunis pour une application embarquée.

CPU / GPU / NPU

Calcul général / calcul parallèle / accélération spécialisée de réseaux.

Flash

Mémoire non volatile : programme et constantes, selon le placement choisi.

SRAM / RAM

Mémoire de travail volatile. Compter les données simultanément vivantes.

SIMD

Une instruction traite plusieurs valeurs ; les formats et alignements comptent.

DMA

Transfert de données piloté par un contrôleur ; bande passante et synchronisation restent limitées.

ISA

Jeu d'instructions du processeur. Un noyau doit exploiter ce qui est réellement disponible.

Ko / Kio

Ko = 1000 octets ; Kio = 1024 octets. Mio = 1024^2 octets dans ce cours.

Lexique · Modèles et expérimentation

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

Paramètre / activation

Un paramètre est appris ; une activation dépend de l'entrée en cours de calcul.

MAC

Multiplier puis accumuler. Préciser la convention avant de comparer des opérations.

Perte / gradient

La perte mesure une erreur ; le gradient indique comment elle varie avec les paramètres.

PTQ / QAT

Quantification après entraînement / apprentissage tenant compte de la quantification.

Cache KV

Clés et valeurs conservées pour éviter certains recalculs pendant la génération.

Runtime / backend

Moteur qui exécute le graphe / implémentation qui choisit les noyaux de la cible.

Latence / débit

Temps d'une réponse / quantité de réponses par unité de temps. Ils ne se confondent pas.

CI / manifeste

Tests et construction automatisés / fiche qui relie version, conditions et fichiers de l'expérience.

Réponses raisonnées · 1/2

L’argument compte davantage que la lettre choisie.

Questions 1 · réponse A · diapositive 6

L’apprentissage peut aider à traiter des variations, à condition de disposer de données et d’un test crédibles. Le choix final peut rester une règle ou une autre interface.

Questions 2 · réponse A · diapositive 11

$h_1=1$ et $h_2=1{,}5$ donnent $\hat{y}=3{,}5$. Les six poids et trois biais donnent neuf paramètres ; les projections réalisent six MAC, hors biais et activations.

Questions 3 · réponse A · diapositive 16

Seule la première proposition respecte les deux exigences. Une passerelle éviterait Internet mais ferait sortir l’audio de l’objet. Un entraînement distant peut rester compatible avec une inférence locale.

Réponses raisonnées · 2/2

L'argument compte davantage que la lettre choisie.

Questions 4 · réponse B · diapositive 23

Le pic annoncé vaut 136 Kio : dépassement de 8 Kio. Dans ce scénario, les poids restent en Flash. Une réutilisation de buffers peut aider si leurs durées de vie ne se recouvrent pas.

Questions 5 · réponse B · diapositive 29

80 ms inchangées + 10 ms pour la brique = 90 ms. Avec 12 ms de conversion ajoutée, 102 ms : l'application ralentit. Les durées seules ne donnent pas l'énergie.

Questions 6 · réponse B · diapositive 34

B satisfait les quatre seuils fictifs. A dépasse la RAM, C manque la qualité requise. Un p95 observé ne garantit pas le pire cas. Tester d'autres conditions peut remettre le choix en cause.